

第3次作业 Sprint 1 迭代报告

姓名：李宗杭

学号：2311451

日期：2026-04-13

项目：校园消费生态智能访谈系统（ISF）

1. 个人承诺的用户故事和设计任务

1.1 Sprint 目标

本次 Sprint 1 我承诺围绕“访谈对话体验”完成 3 条用户故事，确保系统能够：

- 以亲和开场引导学生快速进入访谈；
- 支持自然语言聊天式反馈，不采用问卷选择题；
- 在用户跑题时礼貌拉回主线，保障信息采集效率。

1.2 我负责的用户故事

故事编号	用户故事	SP	优先级
ST-5	作为学生用户，我希望收到 AI 亲和开场引导，明确访谈商铺与主题，从而快速进入对话	1	高 (Must)
ST-6	作为学生用户，我希望用自然语言与 AI 聊天式交互，而不是做选择题，从而轻松提交反馈	3	高 (Must)
ST-8	作为学生用户，我希望跑题时 AI 能礼貌拉回主线，从而保证调研效率与信息质量	3	中 (Must)

1.3 对应的软件设计任务

任务A（ST-5）：开场引导与首问生成

- 任务描述：**点击“开始/重置访谈”后，系统自动生成带有访谈目的和首个问题的开场语。
- 实现原理：**
 - 采用“启动触发 + 首轮引导生成”的两段式机制，确保用户进入页面后能快速获得明确引导；
 - 开场内容由“访谈主题 + 当前阶段目标”共同约束，保证开场话术不偏题；
 - 使用流式输出降低等待感，提升首次交互体验与亲和力。
- 验收标准：**开始访谈后，AI 首条消息应体现“主题明确、语气亲和、可直接作答”。

任务B（ST-6）：自然语言聊天式交互

- 任务描述：**学生通过聊天输入自由表达，系统自动解析并推进访谈，不强制表单填写。
- 实现原理：**
 - 前端仅保留聊天输入，用户可自由叙述，不受固定题目格式约束；
 - 后端通过语义分析将自然语言映射为结构化槽位，形成“表达自由、数据可用”的闭环；

- 槽位归档遵循必填、覆盖、去重等规则，使多轮对话可以持续推进并逐步补齐信息。
- **验收标准：**用户可连续自由输入，系统可持续对话并逐步补齐配置中的必填槽位。

任务C（ST-8）：跑题识别与礼貌拉回

- **任务描述：**当用户跑题或反问时，系统先回应再回到当前阶段目标。
- **实现原理：**
 - 先对用户输入做意图分类，识别“有效回答、跑题、反问、拒答”等情形；
 - 跑题场景采用“礼貌回应 + 主线回收”的双阶段策略，既保持对话温度，又保障访谈效率；
 - 通过过渡态机制实现自动回归主流程，避免会话长期停留在异常分支。
- **验收标准：**用户跑题时，AI 不生硬打断，且下一轮能继续当前阶段信息采集。

2. 软件模块设计报告

2.1 设计方法

采用“**状态机 + 槽位填充 + 提示词工程**”三层协同：

- 状态机负责“对话应该往哪里走”；
- 槽位填充负责“信息是否采集完整”；
- 提示词工程负责“话术是否自然友好”。

2.2 系统结构

1. 前端：Streamlit 聊天界面（消息展示、输入、流式输出）；
2. 分析层：Analyzer 识别用户意图、回答质量、槽位抽取；
3. 控制层：Controller 决策状态迁移（访谈、追问、跑题处理、收尾）；
4. 生成层：Generator 按 `logic_signal` 生成对应语气和策略回复。

2.3 开发环境

项目	选择
OS	Windows
语言	Python 3.10+
框架	Streamlit
LLM SDK	openai>=1.0
数据建模	pydantic>=2.5.0

2.4 技术原理与代码落地

2.4.1 ST-5: 亲和开场引导

- 开场由 `stream_generate_opening()` 生成，避免“黑屏等待”，提升首轮体验；
- 访谈主题来自 `product_info.json` 与当前阶段 `goal`，确保“商铺与调研目标明确”；
- 开场语策略在 `GENERATOR_SYSTEM` 的 `OPENING` 规则中固定，减少回答漂移。

2.4.2 ST-6: 自然语言对话

- 用户输入不受模板限制，直接进入 Analyzer；
- Analyzer 输出结构化 JSON（抽取槽位 + 评估质量 + 意图）；
- DataManager 按 `required`、`type`、`allow_overwrite` 更新数据，保证可持续访谈。

2.4.3 ST-8: 跑题拉回

- Analyzer 识别 `off_topic` 或 `user_question`；
- Controller 优先处理异常分支（优先级高于常规推进）；
- Generator 收到 `OFF_TOPIC_BACK` 后执行“礼貌回应 + 回主线”；
- 回应结束后自动回到 `INTERVIEWING`，维持流程稳定。

2.5 核心代码

2.5.1 ST-5 开场引导（启动后流式开场）

```
def _start_interview() -> None:
    # 重置会话并进入访谈态
    ss = st.session_state
    ss.started = True
    ss.conversation_state = ConversationState.INTERVIEWING
    # 阶段与追问计数器清零
    ss.current_stage_idx = 0
    ss.stage_turn_count = 0
    ss.probe_depth = 0
    # 清空本轮访谈历史与归档数据
    ss.collected_data = {}
    ss.messages = []
    ss.report_md = None
    # 打开“开场白流式输出”开关
    ss.opening_stream_pending = True
    ss.report_stream_pending = False

    if ss.opening_stream_pending:
        # 仅在首次进入访谈时执行一次开场流
        ss.opening_stream_pending = False
        stage = _current_stage()
        # 截取最近对话上下文，帮助模型保持语气连贯
        tail = _chat_tail(ss.messages)
        with st.chat_message("assistant"):
            acc: list[str] = []

            def opening_gen():
```

```

        # 流式生成开场白
        for c in stream_generate_opening(stage.goal, ss.product, stage.slots,
tail):
            acc.append(c)
            yield c

# 在 UI 中流式渲染，并把完整文本写回会话历史
st.write_stream(opening_gen)
ss.messages.append({"role": "assistant", "content": "".join(acc)})

```

2.5.2 ST-6 自然语言聊天交互（输入→分析→归档）

```

if ss.started and ss.conversation_state != ConversationState.FINISHED:
    # 聊天框接收用户自然语言，不做问卷式限制
    if prompt := st.chat_input("请输入您的回答..."):
        # 写入待处理输入，触发 rerun 进入分析流程
        ss.pending_chat_prompt = prompt
        st.rerun()

def _reply_stream_kwargs(user_text: str):
    ss = st.session_state
    stage = ss.outline.stages[ss.current_stage_idx]
    slots = stage.slots

    # LLM 解析：抽取槽位 + 评估质量/意图
    analysis = analyze_user_input(user_text, slots, ss.collected_data)
    # 将解析结果归档到结构化数据（含覆盖/去重规则）
    ss.collected_data = update_collected_data(
        ss.collected_data, analysis.extracted_slots, slots
    )

```

2.5.3 ST-8 跑题识别与礼貌拉回（状态机核心）

```

def update_state(
    current_state: ConversationState,
    evaluation: EvaluationResult,
    advice: AdviceResult,
    stage_turn_count: int,
    probe_depth: int,
    current_stage_idx: int,
    total_stages: int,
    stage_required_filled: bool,
) -> tuple[ConversationState, LogicSignal]:
    # 若用户明确退出，优先进入收尾确认
    if evaluation.user_exit_intent:
        return ConversationState.WRAPPING_UP, LogicSignal.WRAP_CONFIRM

    if current_state in (ConversationState.INTERVIEWING,
ConversationState.PROBING):
        # 跑题/反问优先处理：先回应，再拉回主线
        if evaluation.intent_type in ("off_topic", "user_question"):
            return ConversationState.OFF_TOPIC_HANDLING, LogicSignal.OFF_TOPIC_BACK

```

GENERATOR_SYSTEM = """# Role

你是一位专业、亲和的在线访谈员，正在代表研究团队与真实用户进行结构化访谈。

Strategies（按后端决策信号执行）

- OFF_TOPIC_BACK：用户跑题或反问时：先简短、真诚回应，再明确拉回当前阶段目标。

"""

3. 风险分析

3.1 风险识别（技术/商业/项目）

编号	风险描述	类型	概率P(1-5)	影响I(1-5)	评分 P×I	优先级
R1	LLM 接口超时或失败导致回复中断	技术	4	5	20	1
R2	跑题识别误判，影响对话自然度	技术	4	4	16	2
R3	开场语不稳定，无法清晰说明主题	技术	3	4	12	6
R4	槽位抽取置信度偏低，归档错误	技术	3	5	15	3
R5	配置文件变更后与代码预期不一致	技术	3	4	12	7
R6	学生参与意愿不足，样本量偏低	商业	4	4	16	4
R7	报告结论不被后勤/商铺采纳	商业	3	4	12	8
R8	团队成员时间冲突导致延期	项目	4	4	16	5
R9	多人并行开发出现合并冲突	项目	3	3	9	9
R10	测试样本覆盖不足，线上出现边界问题	项目	3	4	12	10

3.2 重点风险应对（Top 3）

1. R1 LLM 接口不稳定

- 预防：配置 `MOCK_LLM` 作为降级方案；
- 应急：增加重试与超时兜底话术；
- 触发：连续 2 次调用失败。

2. R2 跑题误判

- 预防：补充“用户提问/跑题”样本进行提示词微调；
- 应急：在 `OFF_TOPIC_BACK` 文案中保持宽容语气，降低体验损失。

3. R4 槽位抽取误差

- 预防：保持低置信度过滤阈值（当前 <0.35 丢弃）；
- 应急：对关键必填槽位增加一次确认追问。

4. 个人反思与贡献评估

4.1 个人贡献

- 我本次聚焦“对话体验主链路”，完成 ST-5/ST-6/ST-8 的需求落地分析与任务兑现；
- 对应代码能力已在项目中可运行，且与现有架构一致（状态机 + 槽位填充）。

4.2 改进建议

1. 增加针对跑题、反问、拒答的自动化回归测试集；
2. 为开场语增加可配置模板，便于多商铺/多场景复用；
3. 在报告中补充“已收集槽位覆盖率”，提升结果可信度展示。

5. Github仓库

- 仓库地址：`https://github.com/ReaderQW/ics/tree/sprint1`
- 任务主要涉及文件：`app.py`、`src/controller.py`、`src/analyzer.py`、`src/prompts.py`、`src/data_manager.py`